Database Fundamentals

ACID: Atomicity, Consistency, Isolation, and Durability.

How Databases Implement Atomicity

Databases use two key mechanisms to guarantee atomicity.

1. Transaction Logs (Write-Ahead Logs)

- Every operation is recorded in a **write-ahead log** before it's applied to the actual database table.
- If a failure occurs, the database uses this log to **undo** incomplete changes.

2. Commit/Rollback Protocols

- Databases provide commands like `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK`
- Any changes made between `BEGIN TRANSACTION` and `COMMIT` are considered "in-progress" and won't be permanently applied unless the transaction commits successfully.
- If any step fails, or if you explicitly issue a `ROLLBACK`, all changes since the start of the transaction are undone.

How to Implement Consistency

1. Database Schema Constraints

- **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY**, **CHECK** constraints, and other schema definitions ensure no invalid entries are allowed.

2. Triggers and Stored Procedures

- Triggers can automatically check additional rules whenever rows are inserted, updated, or deleted.

- Stored procedures can contain logic to validate data before committing.

3. Application-Level Safeguards

- While the database enforces constraints at a lower level, applications often add extra checks—like ensuring business rules are followed or data is validated before it even reaches the database layer.

---

3. Isolation

Isolation ensures that concurrently running transactions do not interfere with each other's intermediate states.

Concurrency Anomalies

1. Dirty Read

- Transaction A reads data that Transaction B has modified but not yet committed.
- If Transaction B then rolls back, Transaction A ends up holding an invalid or "dirty" value that never truly existed in the committed state.

(2 and 3) Non repeatable read vs Phantom Read

Summary Comparison

Feature	Non-Repeatable Read	Phantom Read
What changes?	Data within already-read rows.	The number of rows matching a query condition.
Culprit Operations	`UPDATE`, `DELETE`	`INSERT` (primarily), sometimes `DELETE`
Scope	Row-level	Set-level (Range-level)
Prevention Level	Prevented by **Repeatable Read** isolation level.	Prevented by **Serializable** isolation level (the strictest).

Isolation Levels

1. Read Uncommitted

- Allows dirty reads; transactions can see uncommitted changes.
- Rarely used, as it can lead to severe anomalies.

2. Read Committed

- A transaction sees only data that has been committed at the moment of reading.
- Prevents dirty reads, but non-repeatable reads and phantom reads can still occur.

3. Repeatable Read

- Ensures if you read the same rows multiple times within a transaction, you'll get the same values (unless you explicitly modify them).
- Prevents dirty reads and non-repeatable reads, but phantom reads may still happen (depending on the database engine).

4. Serializable

- The highest level of isolation, acting as if all transactions happen sequentially one at a time.
- Prevents dirty reads, non-repeatable reads, and phantom reads.
- Most expensive in terms of performance and concurrency because it can require more locking or more conflict checks.

How Databases Enforce Isolation

1. Locking (Pessimistic Concurrency Control)

- **How it works:** If Transaction A is updating a row, it places an "exclusive lock" on it. If Transaction B wants to read that row, it gets blocked and must wait.
- **The Trade-off:** It provides extremely strong consistency and safety, but at a massive cost to performance. In high-traffic systems, this leads to bottlenecks and "deadlocks" (where Transaction A is waiting for B, and B is waiting for A, causing the system to freeze).

---

2. MVCC (Multi-Version Concurrency Control)

To solve the massive performance bottlenecks caused by pessimistic locking, modern databases (like PostgreSQL and Oracle) use MVCC.

- **The Analogy:** Reading a library book while the author writes a new edition. If you want to read the book, the library gives you the current published copy. Meanwhile, the author is in the back room writing the 2nd Edition.
- **How it works:** Instead of overwriting old data immediately, the database creates a brand new, hidden version of the row with a new timestamp.
    - **Readers** are pointed to the old, safe version of the row.
    - **Writers** work on the new version.
    - Therefore, **readers do not block writers, and writers do not block readers.**

- **The Trade-off:** It drastically increases system throughput and scalability. However, it requires background maintenance. The database ends up with a lot of obsolete "ghost" rows that take up storage space and must be periodically cleaned up (this is what the `VACUUM` process does in PostgreSQL).

---

3. Snapshot Isolation

If MVCC is the *engine* (the mechanism of keeping multiple versions), Snapshot Isolation is the *experience* the database provides to the user based on that engine.

- **The Analogy:** Taking a polaroid photograph of a busy stock market trading floor at exactly 10:00 AM. Even if you stare at that photograph for an hour while the real trading floor goes crazy making millions of trades, the prices in your photograph will never change.
- **How it works:** When a transaction starts, the database notes the exact timestamp. For the entirety of that transaction, the database will use MVCC to only show you the versions of rows that were committed *before* your timestamp.
- **Why it matters:** It completely eliminates the **Non-Repeatable Read** problem. If you query an account balance at the start of your transaction, and then query it again 5 minutes later within the same transaction, you are guaranteed to see the exact same number, regardless of what other transactions have done in the background.

---

4. Durability

Durability ensures that once a transaction has been committed, the changes it made will survive, even in the face of power failures, crashes, or other catastrophic events.

How Databases Ensure Durability

1. Transaction Logs (Write-Ahead Logging)

1. **Write Changes to WAL**: The intended operations (updates, inserts, deletes) are recorded in the WAL on durable storage (disk).
2. **Commit the Transaction**: Once the WAL entry is safely persisted, the database can mark the transaction as committed.
3. **Apply Changes to Main Data Files**: The updated data eventually gets written to the main files—possibly first in memory, then flushed to disk.

If the database crashes, it uses the WAL during **recovery**:

- **Redo**: Any committed transactions not yet reflected in the main files are reapplied.
- **Undo**: Any incomplete (uncommitted) transactions are rolled back to keep the database consistent.

2. Replication / Redundancy

- **Synchronous Replication**: Writes are immediately copied to multiple nodes or data centers. A transaction is marked committed only if the primary and at least one replica confirm it’s safely stored.
- **Asynchronous Replication**: Changes eventually sync to other nodes, but there is a (small) window where data loss can occur if the primary fails before the replica is updated.

3. Backups

Regular **backups** provide a safety net beyond logs and replication. In case of severe corruption, human error, or catastrophic failure:

- **Full Backups**: Capture the entire database at a point in time.
- **Incremental/Differential Backups**: Store changes since the last backup for faster, more frequent backups.
- **Off-Site Storage**: Ensures backups remain safe from localized disasters, allowing you to restore data even if hardware is damaged.

SQL vs NoSQL

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Data Model	Structured, table-based with fixed schemas and relationships	Flexible data models (document, key-value, column-family, graph)
Schema	Rigid; must be predefined with strict structure	Schema-less or flexible; allows dynamic and unstructured data
Scalability	Vertical scaling (scale-up)	Horizontal scaling (scale-out) across distributed nodes
Consistency	Strong consistency with ACID transactions	Eventual consistency, though tunable in some systems (e.g., MongoDB)
Query Language	SQL (standardized and powerful for complex queries)	Varies (e.g., MongoDB query language, Cassandra Query Language)
Transaction Support	Full ACID (Atomicity, Consistency, Isolation, Durability)	BASE model (Basically Available, Soft state, Eventually consistent); some support ACID in limited contexts
Performance	Optimized for complex queries and relationships, slower for large-scale writes	Optimized for high-throughput reads/writes, especially in distributed systems
Use Cases	Best for structured data and applications needing strong consistency (e.g., financial systems, ERP, CRM)	Best for unstructured/semi-structured data and large-scale distributed systems (e.g., big data, real-time analytics, social media)
Scaling Challenges	Difficult to shard or partition across servers	Built to shard and distribute data easily
Data Integrity	High; enforces strong data integrity through constraints and relationships	Varies; integrity is often managed at the application level
Joins and Relationships	Supports complex joins between tables and foreign key relationships	Relationships are either embedded (documents) or handled differently (e.g., graph databases)
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, DynamoDB, Redis, Neo4j

If your application requires structured data, complex queries, and transaction management, an SQL database is likely the best choice.

However, if your application demands scalability, flexibility, and the ability to handle unstructured data, a NoSQL database may be more suitable.

Sharding

3. How Does Sharding Work?

The sharding process involves several key components including:

1. **Sharding Key**: The shard key is a unique identifier used to determine which shard a particular piece of data belongs to. It can be a single column or a combination of columns.
2. **Data Partitioning**: splitting the data into shards based on the shard key.
3. **Shard Mapping**: Creating a mapping of shard keys to shard locations.

4. Sharding Strategies

Hash-Based Sharding

Data is distributed using a hash function, which maps data to a specific shard.

- **Example:** `Hash(user_id) % 2` determines the shard number for a user, distributing users evenly across 2 shards.

Range-Based Sharding

Data is distributed based on a range of values, such as dates or numbers.

- **Example:** Shard 1 contains records with IDs from 1 to 10000, Shard 2 contains records with IDs from 10001 to 20000, and so on.

Geo-Based Sharding

Data is distributed based on geographic location.

- **Example:** Shard 1 serves users in North America, Shard 2 serves users in Europe, Shard 3 serves users in Asia.

Directory-Based Sharding

Maintains a lookup table that directly maps specific keys to specific shards.

6. Best Practices for Sharding

- **Choose the Right Sharding Key.**

- **Use Consistent Hashing**
- **Monitor and Rebalance Shards**: Regularly monitor shard performance and data distribution. Rebalance shards as needed to ensure optimal performance and data distribution.
- **Handle Cross-Shard Queries Efficiently**: Optimize queries that require data from multiple shards by using techniques like query federation or data denormalization.

REPLICATION

Two metrics define recovery quality. Recovery Point Objective (RPO) measures how much data loss is acceptable, specifically how far back you'd need to roll back after a failure.

Recovery Time Objective (RTO) measures how long the system can be offline before the business impact becomes unacceptable.

HLD Interview Question Pool & Required Depth

1. Core Concepts & Trade-offs

Q: "We need to expand our database to a new region. Should we use Synchronous or Asynchronous replication?"

- **What is asked:** The interviewer wants to see if you understand the CAP theorem implications, latency costs, and data consistency trade-offs across large geographic distances.
- **Depth required: Deep.** You must confidently explain that Synchronous replication across regions is generally an anti-pattern because the round-trip network latency will block write operations. You must suggest Asynchronous replication across regions while acknowledging the trade-off: a small consistency window (replication lag) where a failover could result in data loss.

Q: "Why can't we just rely on our nightly backups instead of setting up database replication?"

- **What is asked:** The difference between disaster recovery mechanisms and continuous availability.
- **Depth required: Moderate.** You need to define RPO (Recovery Point Objective - how much data you can afford to lose) and RTO (Recovery Time Objective - how long you can afford to be down). Backups measure RPO in hours/days and RTO in hours. Replication measures RPO in milliseconds/seconds and RTO in minutes.

2. System Performance & Scaling

Q: "Our primary database is bottlenecking due to heavy read traffic. How would you solve this using replication?"

- **What is asked:** Understanding of Read Replicas, separation of Read/Write concerns, and the routing logic required.
- **Depth required: Moderate to Deep.** You should suggest setting up asynchronous read replicas. However, an elite answer will address the immediate edge case: **Replication Lag**. You need to explain how you would handle scenarios where a user updates their profile and immediately refreshes the page, but the read replica hasn't received the update yet (e.g., implementing "Read-Your-Own-Writes" consistency by routing reads from the primary for a few seconds after a write).

3. Advanced Architectures & Conflict Resolution

Q: "How would you design a system where users in the US and users in India can both write to the database locally with minimal latency?"

- **What is asked:** Knowledge of Active-Active (Multi-Primary) Geo-Distribution.
- **Depth required: Deep.** You must mention that both nodes will accept reads and writes, eliminating cross-region write latency. Crucially, you *must* discuss **Conflict Resolution**. What happens if a US user and an India user edit the same record at the same millisecond? You should be able to discuss resolution strategies like Application-level resolution, Last-Write-Wins (LWW) using timestamps, or utilizing CRDTs (Conflict-free Replicated Data Types) to automatically merge concurrent changes.

4. Data Capture & State

Q: "How exactly do changes get from the primary database to the replicas without degrading the primary's performance?"

- **What is asked:** Understanding of underlying database mechanics, specifically Change Data Capture (CDC) and transaction logs.
- **Depth required: Moderate.** You don't need to write the code for it, but you must know that databases don't just "copy tables" constantly. You should explain that replication relies on reading the primary database's Write-Ahead Log (WAL) or transaction log. For a new replica, the system takes an initial full snapshot, and then incrementally streams the transaction logs to catch up.

5. Data Locality & Cost Optimization

Q: "Replicating our entire global database to every regional office is becoming too expensive. How can we optimize this?"

- **What is asked:** Full vs. Partial database replication.
- **Depth required: Moderate.** You should discuss Partial Replication. Explain that a central/HQ database can hold all records, but regional databases only replicate data relevant to their local users. You should also acknowledge the trade-off: this adds complex routing logic at the application layer, as the app now needs to know *which* database to query if a user travels or requests cross-regional data.

What is data replication?

Data replication is the process of keeping copies of the same data in multiple locations so apps stay available, reliable, and responsive. Without it, a single database failure can make an entire application unavailable — there's no copy to fall back to, and recovery depends entirely on how recent your last backup was. Replication changes that by keeping one or more replicas current as writes happen, so the system can fail over without waiting for a restore.

What's the difference between replication & backup?

Replication keeps copies current continuously; backup stores point-in-time snapshots for later recovery. Most production architectures rely on both.

Replication handles availability during an incident — if a primary goes down, a replica takes over. Backup handles recovery after one — if data is corrupted or deleted, you restore from a known-good snapshot.

When should you use synchronous vs. asynchronous replication?

Synchronous replication works best when you need tighter consistency and network latency between nodes is low, typically within the same region. Asynchronous replication is the practical choice for cross-region deployments, where waiting for replica confirmation would add too much write latency. Many teams run synchronous replication within a region and asynchronous replication across regions, accepting a small consistency window at the geographic boundary in exchange for lower write latency globally.

What is Active-Active Geo Distribution?

Active-Active Geo Distribution is a multi-region replication model, available in Redis Cloud and Redis Software, where every node accepts both reads and writes and changes synchronize across regions automatically using CRDTs. It's a good fit when your users are globally distributed and cross-region write latency is a real problem. The trade-off is added architectural complexity and the need to think carefully about data type selection, since some types fall back to last-write-wins rather than merge-based conflict resolution.

What are the biggest challenges with data replication?

The most common challenges are infrastructure cost, replication lag, conflict resolution, and bandwidth consumption at scale. Lag and conflict resolution require the most deliberate upfront decisions: lag means reads may return stale data immediately after a write, and conflict resolution determines what happens when two nodes update the same record before synchronizing.

Bloom Filters

How Does a Bloom Filter Work?

A Bloom filter works by using multiple hash functions to map each element in the set to a bit array.

1. Initialization:

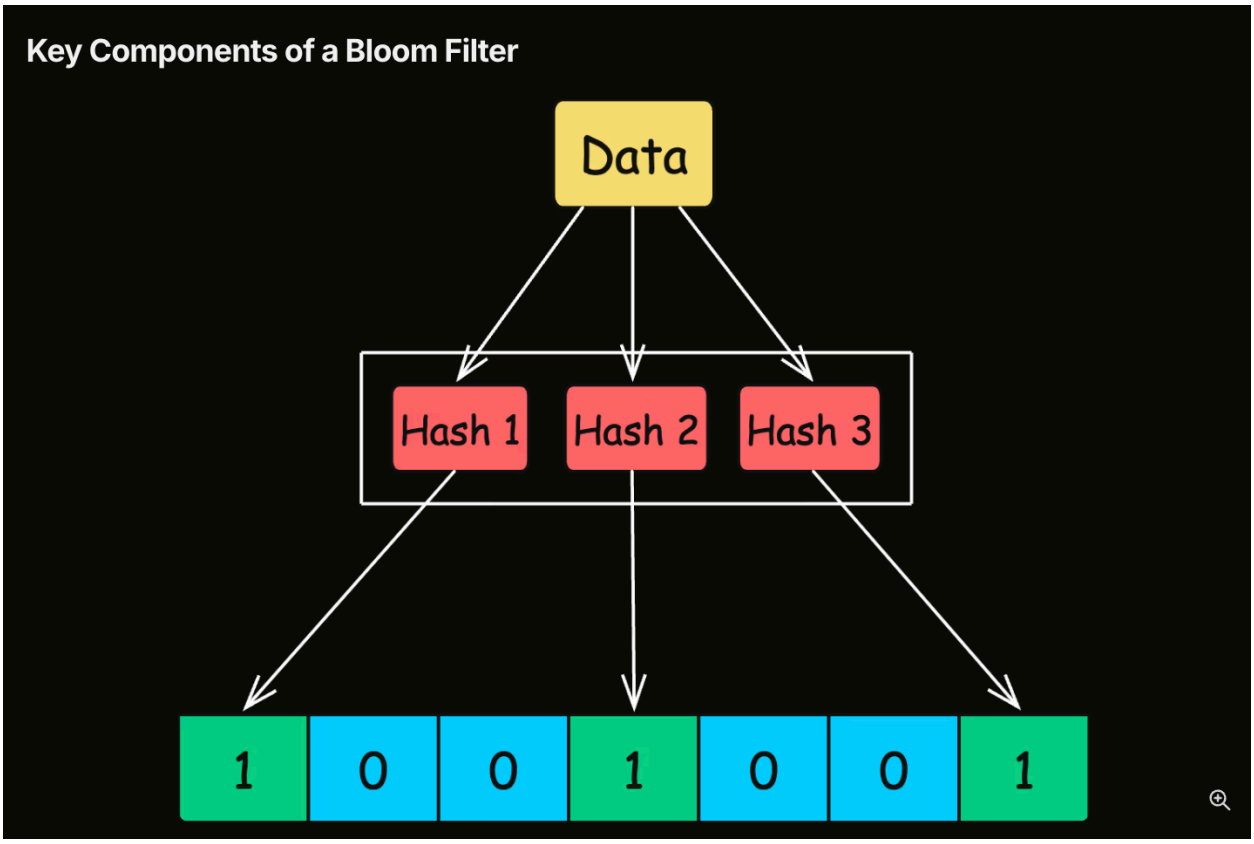
- A Bloom filter starts with an empty bit array of size `m` (all bits are initially set to 0).
- It also requires `k` independent hash functions, each of which maps an element to one of the `m` positions in the bit array.

2. Inserting an Element:

- To insert an element into the Bloom filter, you pass it through each of the `k` hash functions to get `k` positions in the bit array.
- The bits at these positions are set to 1.

3. Checking for Membership:

- To check if an element is in the set, you again pass it through the `k` hash functions to get `k` positions.
- If all the bits at these positions are set to 1, the element is considered to be in the set (though there's a chance it might be a false positive).
- If any bit at these positions is 0, the element is definitely not in the set.




```
public BloomFilter(int size, Function<String, Integer>... hashFunctions) {
    this.size = size;
    this.bitArray = new BitSet(size);
    this.hashFunctions = hashFunctions;
}

// Method to add an element to the Bloom Filter
public void add(String item) {
    for (Function<String, Integer> hashFunction : hashFunctions) {
        int index = Math.abs(hashFunction.apply(item) % size);
        bitArray.set(index);
    }
}

// Method to check if an element is possibly in the Bloom Filter
public boolean mightContain(String item) {
    for (Function<String, Integer> hashFunction : hashFunctions) {
        int index = Math.abs(hashFunction.apply(item) % size);
        if (!bitArray.get(index)) {
            return false; // If any bit is 0, the item is definitely not in the set
        }
    }
```

```
    return true; // All bits are 1, so the item is probably in the set
}
```

5. Real-World Applications of Bloom Filters

1. Web Caching

Problem: Web servers often cache frequently accessed pages or resources to improve response times. However, checking the cache for every resource could become costly and slow as the cache grows.

Solution: A Bloom Filter can be used to quickly check if a URL might be in the cache. When a request arrives, the Bloom Filter is checked first. If the Bloom Filter indicates the URL is “probably in the cache,” a cache lookup is performed.

If it indicates the URL is “definitely not in the cache,” the server skips the cache lookup and fetches the resource from the primary storage, saving time and resources.

2. Spam Filtering in Email Systems

Problem: Email systems need to filter out spam emails without constantly checking large spam databases.

Solution: A Bloom Filter can store hashes of known spam email addresses. When a new email arrives, the Bloom Filter checks if the sender's address might be in the spam list.

This allows the email system to quickly determine whether an email is likely to be spam or legitimate.

3. Databases

Problem: Databases, especially distributed ones, often need to check if a key exists before accessing or modifying data. Performing these checks for every key directly in the database can be slow.

Solution: Many databases, such as **Cassandra**, **HBase**, and **Redis**, use Bloom Filters to avoid unnecessary disk lookups for non-existent keys. The Bloom Filter quickly checks if a key might be present. If the Bloom Filter indicates “not present,” it can skip the database lookup.

4. Content Recommendation Systems

Problem: Recommendation systems, such as those used by streaming services, need to avoid recommending content that users have already consumed.

Solution: A Bloom Filter can track the content each user has previously watched or interacted with. When generating new recommendations, the Bloom Filter quickly checks if an item might already have been consumed.

5. Social Network Friend Recommendations

Problem: Social networks like Facebook or LinkedIn recommend friends or connections to users, but they need to avoid recommending people who are already friends.

Solution: A Bloom Filter is used to store the list of each user’s existing connections. Before suggesting new friends, the Bloom Filter can be checked to ensure the user isn’t already connected with them.

6. Limitations of Bloom Filters

1. False Positives

Bloom Filters can produce false positives, meaning they may incorrectly indicate that an element is present in the set when it is not.

For instance, in a database system, this might trigger unnecessary cache lookups or wasted attempts to fetch data that doesn’t actually exist.

2. No Support for Deletions

Standard Bloom Filters do not support element deletions. Once a bit is set to 1 by adding an element, it cannot be unset because other elements may also rely on that bit.

Variants like the **Counting Bloom Filter** can allow deletions by using counters instead of bits, but this requires more memory.

Indexing

Data structures

1. B-Trees: The "Read-Optimized" Standard

B-Trees (and their more common variant, **B+ Trees**) are the backbone of almost all Relational Databases (MySQL, PostgreSQL).

How they work:
A B-Tree is a "balanced" tree. It keeps data sorted and allows for searches, sequential access, insertions, and deletions in logarithmic time (o(logn))
• **Fixed-Size Pages:** The database breaks the data into fixed-size blocks called "pages" (usually 4KB or 8KB).
• **The Root and Internal Nodes:** These act as signposts. They contain a range of values and pointers to child pages.
• **Leaf Nodes:** In a B+ Tree, the actual data (or pointers to rows) lives only in the bottom-most "leaf" nodes. These leaves are linked together in a chain, which makes "range scans" (e.g., *give me all prices between $10 and $50*) incredibly fast.

The Problem with B-Trees (The "Write" Tax):
When you update a B-Tree, the database has to find the specific page on the disk, overwrite it, and potentially "rebalance" the tree. This involves **Random I/O**—the disk head has to jump around to different locations. Random writes are the slowest operation a disk can perform.

2. LSM Trees: The "Write-Optimized" Alternative

LSM (Log-Structured Merge-Trees) are used in NoSQL databases like Cassandra, LevelDB, and RocksDB. They are designed for systems that handle massive amounts of incoming data (like logging, or Twitter feeds).

How they work:
Instead of trying to find and update a specific spot on the disk (Random I/O), an LSM Tree turns every update into a **Sequential Write**.
1. **MemTable:** When data comes in, it is first written to a sorted buffer in RAM called a MemTable.
2. **SSTables (Sorted String Tables):** Once the MemTable is full, the database freezes it and flushes it to the disk as a single, sorted file. This is a sequential write, which is extremely fast.
3. **Compaction:** Over time, you end up with many small SSTable files on your disk. A background process called "Compaction" runs to merge these files, keep them sorted, and delete old versions of data.

The Problem with LSM Trees (The "Read" Tax):
Because the data could be in the MemTable or spread across several SSTables on the disk, a "Read" might have to check multiple files to find the latest version of a record. This is called **Read Amplification**. To fix this, LSM Trees use **Bloom Filters** (a fast way to check if a key *might* exist in a file without actually reading it).

Indexes

1. Composite Indexes & The Left-Prefix Rule

A **Composite Index** is an index created on multiple columns at once. For example, in a `Users` table, you might create an index on `(Last_Name, First_Name)`.

How it works:

The database sorts the index first by the first column, and **only then** by the second column for any matching first values.

Analogy: A Phonebook.
A phonebook is a composite index on `(Last_Name, First_Name)`.

- All the "Smiths" are grouped together.
- Within the "Smiths," the "Andrews" come before the "Zachs."

The Left-Prefix Rule:

This rule states that a composite index can only be used if the query includes the **leftmost** column(s) of the index.

- **Query 1:** `WHERE Last_Name = 'Smith'`
  - ✅ **Works.** You go to the 'S' section of the phonebook.
- **Query 2:** `WHERE Last_Name = 'Smith' AND First_Name = 'John'`
  - ✅ **Works.** You go to 'Smith', then look for 'John'.
- **Query 3:** `WHERE First_Name = 'John'`
  - ❌ **Fails.** The phonebook is useless if you only know a first name. You'd have to read the whole book (Full Table Scan).

Interview Tip: Always order your composite index columns based on which ones appear most frequently in your `WHERE` clauses.

---

2. Covering Indexes

A **Covering Index** is a special case where the index itself contains **all** the information the query needs.

The "Double Lookup" Problem:

Normally, a database uses an index to find a pointer, then goes to the actual table on the disk to get the rest of the data. This is two steps (Index Read + Table Read).

The Solution:

If your index includes every column mentioned in your `SELECT` and `WHERE` clauses, the database **never touches the actual table.**

Example:

- **Table:** `Orders` (Order_ID, Date, Customer_ID, Total_Amount)
- **Index:** `(Customer_ID, Total_Amount)`
- **Query:** `SELECT Total_Amount FROM Orders WHERE Customer_ID = 505;`

Because the `Total_Amount` is physically stored *inside* the index structure alongside the `Customer_ID`, the database answers the query directly from the index.

Why this matters for HLD:
Covering indexes are a massive performance boost because they reduce **Disk I/O**. Reading a small index from memory is 100x faster than fetching heavy data rows from the disk.

Level 1: Fundamentals (Mechanics)

1. Explain the difference between a Clustered and a Non-Clustered index.

- **Clustered Index:** This index defines the physical order in which data is stored on the disk. Because data can only be stored in one order, there is only **one** clustered index per table (usually the Primary Key).
- **Non-Clustered Index:** This is a separate structure from the data rows. It contains the indexed columns and a pointer (bookmark) to the actual data row. You can have many of these.
- **Analogy:** A Clustered Index is like a Dictionary (the words themselves are stored alphabetically). A Non-Clustered Index is like the Index at the back of a textbook (the index is separate from the content, pointing you to a page number).

2. What happens to my write performance when I add five different indexes to a table?

- **Answer:** Write performance decreases. Every time you `INSERT`, `UPDATE`, or `DELETE` a row, the database must also update all five index structures to keep them synchronized. This increases disk I/O and latency. In HLD, the trade-off is: "Faster reads at the cost of slower writes."

3. What is a "Full Table Scan", and why is it the enemy of scalability?

- **Answer:** A Full Table Scan occurs when the database engine has to examine every single row in a table because no suitable index exists.
- **Scalability Issue:** It is an $O(N)$ operation. As your data grows from 1 million to 1 billion rows, the query time grows linearly, eventually timing out and consuming all CPU/memory resources.

Level 2: Strategic Trade-offs (Decision Making)

4. We are designing a logging system with 1 million writes per second. Would you add an index on the "Error Message" column?

- **Answer:** Likely **No**. In a high-throughput write-heavy system, indexing a long string like an error message would create a massive bottleneck.
- **Alternative:** Use an **LSM Tree-based storage** (like Cassandra) or move the logs to a dedicated search engine (like Elasticsearch) where indexing happens asynchronously, so it doesn't block the ingestion of logs.

6. How do you decide which columns to index in a complex Search feature?

- **Answer:** Analyze the **Query Patterns**. Identify the `WHERE` clause filters, `ORDER BY` columns, and `JOIN` keys.
- **Depth:** I would use **Composite Indexes** for frequent multi-column filters, ensuring I follow the **Left-Prefix Rule**. I would also consider a **Covering Index** for the most critical high-traffic queries to avoid the "Double Lookup" (reading the index then the table).

Level 3: Distributed Systems (Architecture)

7. If we shard our database by `User_ID`, how do we handle a query that searches by `Email`?

- **Option A (Local Index):** Every shard has its own index. You must "Scatter-Gather"—query every single shard and aggregate the results. This is slow and doesn't scale well.

Option A: Scatter-Gather (The "Run Around" Method)

In this approach, you don't have a master list.

1. You open **Cabinet 1** and look for the email. Not there.
2. You open **Cabinet 2**. Not there.
3. You open **Cabinet 3**. Not there.
4. You open **Cabinet 4**. Found it!

Why it's bad: If you have 1,000 cabinets (shards) instead of 4, you have to search 1,000 places for **one** simple query. This uses a massive amount of CPU and time. This is called **Scatter-Gather**.

- **Option B (Global Index/Mapping Table):** Create a separate service or table that maps `Email -> User_ID`. You query this first to find the correct shard, then go directly to that shard. This is the standard HLD approach for high-scale systems.

Option B: Global Index / Mapping Table (The "Directory" Method)

To fix this, you create a tiny, separate "Directory" (a Mapping Table) that is organized by **Email**.

Email	User_ID
alex@email.com	305
beatrice@email.com	12
charlie@email.com	150

Export to Sheets

How the search works now:

1. **Step 1:** You look at the **Directory**. It tells you `alex@email.com` has **User_ID 305**.
2. **Step 2:** You know User_ID 305 is in **Cabinet 4**.
3. **Step 3:** You go directly to Cabinet 4 and get the full profile.

Why it's the "Standard" approach:

Even if you have 1 million cabinets, you only ever perform **two quick lookups**:

1. Check the Directory (Mapping Table).
2. Go to the specific Shard.

Do say: "I would use a **Global Secondary Index** or a **Mapping Table**. This acts as a lookup service to translate the secondary attribute (Email) into the shard key (User_ID), avoiding an expensive scatter-gather across all shards."**8. What do you do if your index becomes too large to fit in RAM?**

- **Answer:**
    1. **Vertical Scaling:** Increase RAM (expensive and limited).

2. **Horizontal Sharding:** Split the data across multiple nodes so each node's index fits in its own RAM.
3. **Use a B+ Tree:** B+ Trees are designed to live on disk. While RAM is faster, B+ Trees minimize disk seeks (IOPS) by having high "fan-out," making them viable even if they aren't fully in memory.
4. **Index Compression:** Use prefix compression or other techniques to reduce the footprint.

9. Why use LSM Trees over B+ Trees for write-heavy NoSQL?

- **Answer:** B+ Trees require **Random Writes** (updating various parts of the tree on disk), which are slow. LSM Trees (Log-Structured Merge-Trees) buffer writes in memory (MemTable) and then flush them to disk as **Sequential Writes** (SSTables). Sequential I/O is significantly faster than random I/O, making LSM Trees superior for ingestion-heavy systems.

Types of Databases

"If we start with SQL and our traffic grows 100x, at what point do we decide to migrate to NoSQL?"

- **Answer:** Migrate when the cost of vertical scaling becomes prohibitive, or when the relational schema becomes a bottleneck for development speed due to frequent "Alter Table" operations on billions of rows.

---

1. The Global Leaderboard (Redis Sorted Sets)

The Core Question: *"How would you design a real-time leaderboard for a game with 10 million active players where rank updates instantly?"*

- **The Answer:** Use **Redis Sorted Sets (ZSETs)**. Internally, Redis uses a **Skip List** and a **Hash Map**. This allows you to add/update scores and retrieve a user's rank in $O(\log n)$ time.
- **MNC Variant:** *"What if the leaderboard needs to be 'Regional' (India, US, Europe) but also 'Global'?"*

- **Pro Tip:** Maintain separate ZSETs for each region and use `ZUNIONSTORE` to aggregate them for the global view, or update both the regional and global ZSETs simultaneously (write-through).

- **MNC Variant:** *"How do you handle 'Tie-breaking' if two players have the exact same score?"*
  - **Pro Tip:** Use a composite score. Instead of just `Score`, use `Score.Timestamp`. This ensures the person who reached the score first is ranked higher.

---

2. Proximity & Ride-Hailing (Geospatial Databases)

The Core Question: *"How would you find the 10 nearest drivers for a rider in Uber or the nearest restaurants in DoorDash?"*

- **The Answer:** Use a database with **Geospatial Indexing** (Redis Geo, MongoDB, or PostGIS). These use **Quad-trees** or **Geohashes**.
- **MNC Variant:** *"Why not just use a standard SQL query with the Pythagorean theorem ($a^2 + b^2 = c^2$)?"*
  - **Pro Tip:** A standard SQL query requires a "Full Table Scan," which is $O(n)$. For millions of drivers, this is too slow. Geospatial indexes divide the world into a grid; you only search the "cells" surrounding the user, reducing the search space to $O(\log n)$.

---

3. Social Graph & Fraud Detection (Graph Databases)

The Core Question: *"How do you suggest 'People You May Know' or detect a ring of fake accounts sending money to each other?"*

- **The Answer:** Use a **Graph Database** like Neo4j or AWS Neptune. In RDBMS, finding a "friend of a friend of a friend" requires three expensive `JOIN` operations. In a Graph DB, this is a simple **Graph Traversal** (following pointers), which stays fast even as the dataset grows.
- **MNC Variant:** *"Can we use a Relational DB for this? What is the trade-off?"*
  - **Pro Tip:** You can use Recursive Common Table Expressions (CTEs) in SQL, but the performance degrades exponentially with each "hop." If the relationship is the primary data point (not the entity), go with Graph.

---

4. Distributed Logging & IoT (Wide-Column Stores)

The Core Question: *"You are receiving 1 million sensor readings per second from smart home devices. How do you store them for later analysis?"*

- **The Answer:** Use a **Wide-Column Store** like Cassandra or ScyllaDB. These use **LSM Trees** (Log-Structured Merge-Trees) which turn random writes into sequential disk writes, making them incredibly fast for "Write-Heavy" workloads.
- **MNC Variant:** *"How do you handle data 'TTL' (Time To Live) so the database doesn't grow infinitely?"*
    - **Pro Tip:** Cassandra allows you to set a TTL at the row level. Once the time expires, the database marks the data with a "Tombstone" and deletes it during the next compaction cycle.

---

5. Similarity Search & AI (Vector Databases)

The Core Question: *"How do you build a recommendation engine that finds products 'similar' to what a user just bought, using AI embeddings?"*

- **The Answer:** Use a **Vector Database** (Pinecone, Milvus, or Weaviate). Unlike traditional DBs that look for exact matches, Vector DBs use **Approximate Nearest Neighbor (ANN)** algorithms to find items that are "close" in a multi-dimensional mathematical space.
- **MNC Variant:** *"How is this different from a 'Search Engine' like Elasticsearch?"*
    - **Pro Tip:** Elasticsearch is for **Keyword** search (exact words). Vector DBs are for **Semantic** search (meaning). If I search for "King," a keyword search finds the word "King"; a Vector search might find "Royalty" or "Throne."

---

6. Summary Table for "Pro" Scenarios

Use Case	Recommended DB	"Pro" Justification
Stock Market Tickers	InfluxDB / Kdb+	Optimized for time-ordered data and high-speed ingestion.

Autocomplete / Fuzzy Search	Elasticsearch	Uses Inverted Indices for fast text matching.
Distributed Locks / Config	Etcd / Zookeeper	Uses **Paxos** or **Raft** consensus to guarantee 100% consistency.
Large File Storage	S3 / GCS (Object Store)	Databases are bad at storing large blobs (images/videos); store the link in DB and file in S3.

The Final "MNC-Style" Curveball:

"If you had to build a system that needs both Graph-like relationships and High-speed caching, would you use two databases or one?"

- **The Pro Answer:** "I would use a **Polyglot Persistence** architecture. No single database is perfect for everything. I'd use Neo4j for the relationship mapping and Redis for the caching layer, ensuring they stay in sync via a Change Data Capture (CDC) mechanism."